

Multi-Account (Multi-Tenant) Helix OTA — Phased Delivery Plan (§11.4.167 / §11.4.172)

Revision: 1 Last modified: 2026-07-10T11:18:54Z

Scope. This is the phased DELIVERY PLAN for adding an ACCOUNT (tenant) layer to the Helix OTA control plane (server/, Go/Gin modular monolith) and every client surface, delivered as an **isolated big-feature work-stream** (§11.4.167) with a **living production-readiness plan** (§11.4.172). It synthesizes the authoritative design set — it does **not** invent conflicting content. Reading order for anyone resuming: 00_INDEX.md (mandate + open questions) → 10_/11_/12_* (as-is discovery) → 20_target_multitenancy_data_model.md (SSOT data model) → 21_authz_rbac_superadmin.md → 22_api_surface.md → 23_ui_ux_all_surfaces.md → 24_project_side_cli.md → 25_device_side_update_client.md → 26_security_threat_model.md → 27_test_strategy.md → this doc.

This plan is APPROVAL-GATED (§11.4.6 / §11.4.66). No production code is written under this directive until the operator approves the plan AND resolves the §5 decision gate. Every milestone cites the design doc(s) it derives from. Every estimate is marked as an estimate (§11.4.6 — no false precision). Nothing here is a §11.4 completion claim: the whole feature is a to-be proposal across 20_*–27_*.

1. Work-stream setup — isolated big-feature lifecycle (§11.4.167 / §11.4.179 / §11.4.181)

This is a BIG feature (a new tenancy layer touching the data model, authZ, every OTA route, two SPAs, a new CLI binary, the device agent, security, and object storage). Per §11.4.167 it develops as its **own isolated feature work-stream**, kept separate from trunk until operator-approved after a full retest, with trunk merged in regularly.

1.1 Canonical branch (one name, everywhere — §11.4.181)

One feature maps to **exactly ONE canonical branch name used identically on the main repo AND every touched owned submodule** (§11.4.181):

- **Branch:** feature/multi-tenant-accounts (lowercase kebab per §11.4.29).
- **Touched owned submodules** (each branches under the SAME name when first touched, never a per-repo variant): submodules/ota-android-agent (M6 — the net-new setup wizard UI + the concrete ControlPlaneClient that 11_* \$Honest-gaps 2 / 25_* \$5 say is absent) and the **containers submodule** (§11.4.76/§11.4.161 — new rootless- podman compose recipes for the M7 object-storage MinIO dev backend + the 27_*-\$0.1 live-PostgreSQL RLS test topology). An untouched submodule stays on its base branch. server/cmd/helix-ota (M5 CLI) is in the main repo, not a submodule.
- The canonical name is recorded ONCE in the claim/ logic_group-branch registry (§11.4.176/§11.4.181) and looked up, never re-invented. A recommended gate CM-BRANCH-NAME-CONSISTENCY fails on any divergent submodule branch name.

1.2 Corruption-isolated, CoW-cheap checkout (§11.4.167 / §11.4.179)

- The work-stream develops in an **independent repo with its OWN .git** (own object store, own index, own lock namespace) — NOT a git worktree sharing one common .git (§11.4.179: a shared common-dir is a single point of failure — one stale lock freezes every stream). Where disk-feasibility is a concern, use the §11.4.167 D reflink/CoW-clone-with-own-.git path (btrfs/XFS/ZFS/APFS

reflink shares extents on disk while keeping a per-stream .git), so isolation + disk-feasibility both hold.

- Regular **trunk-merge-in** cadence (§11.4.167): main is merged into the feature branch at each milestone boundary and on any material trunk change, integrated by the §11.4.113 merge-onto-latest-main procedure — **force-push is strictly forbidden** (§11.4.113); every commit/push wrapper auto-reaps only provably-dead locks (§11.4.180) and fans out to all four upstreams (§2.1).

1.3 Release identity + operator-approval merge gate

- **Release-tag / version naming (§11.4.151):** tags on the main repo AND every touched owned submodule are prefixed helix_ota-<version> (prefix resolved from HELIX_RELEASE_PREFIX in .env — the tracked .env.example:21 documents
HELIX_RELEASE_PREFIX=helix_ota; else the lowercased project-root dir name helix_ota). The same prefix is greppable across main + submodules for one release. No tags exist yet (verified this session — greenfield naming).
 - **Operator-approval merge gate (§11.4.167 / §11.4.126).** The feature branch merges to main ONLY after: (a) the §5 decision gate is resolved; (b) the M8 full-suite retest (§11.4.40) is GREEN with captured evidence; (c) an independent code review reaches a zero-finding GO (§11.4.125/§11.4.134/§11.4.142); (d) the operator explicitly approves; and (e) — because a release is the terminal condition — the §11.4.185 **manual QA-team confirmation** lands as the FINAL step. The submodule branch/tag cascade (§11.4.167) merges + tags each touched submodule under the same name/prefix in the same window.
 - **Build + test discipline (§11.4.167):** single-builder + per-device exclusive test queues; builds run via the containers submodule where applicable (§11.4.173); the server go build ./... / gofmt / go vet stay clean (project MANDATORY constraint).
-

2. Phased milestones (dependency-ordered)

Eight milestones, dependency-ordered. Each cites its design doc(s), the 27_* tests that prove it, and a captured-evidence exit criterion (§11.4.69 — every PASS cites an artefact under docs/qa/<run-id>/, never a metadata-only PASS). **One cross-cutting reorder is called out in §3 and honored here: the object-storage seam (M7) is a prerequisite for the byte-level e2e layers of M5/M6** — those milestones' control-

plane work lands on schedule, but their “device downloads + applies the exact bytes” e2e proof is SKIP-with-reason: `object_storage_absent` until M7’s Storage seam lands (24_* §4, 25_* §5, 27_* §0.2).

M1 — Data model + migrations + `store.Repository` extension · derives from 20_* (+ 10_* as-is)

- **Scope.** The keystone. Add the four new/lifted entities — Account, persisted User (closes the 10_* §3 no-persisted-user gap), Membership, and the minimal per-account Role skeleton — and scope the existing OTA model (Device/Artifact/Release/Deployment/Telemetry/Group/Audit) with a denormalized `account_id` (RLS key) + `project_id` (20_* §1–§2). Tenancy model: **shared schema + `account_id` column + PostgreSQL RLS** (20_* §0).
- **Deliverables.** (a) New `store.Repository` methods mirroring the Projects block — Accounts CRUD, the User CRUD that does not exist today, Memberships, and the atomic `CreateAccountWithOwner` seam that closes the orphan-tenant hole (20_* §3.1/§3.3), implemented by BOTH `MemoryRepository` and `PostgresRepository`. (b) Scope the existing accessors — explicit `accountId` param on single-entity get/create/update + `AccountID/ProjectID` fields on the `*Filter` structs (20_* §3.2 hybrid). (c) The additive migration `002_accounts_multitenancy` (opaque TEXT ids to mirror the shipped store; UUID in the canonical 001-style schema) + the 003 NOT-NULL/uniqueness/composite-FK/index/RLS follow-up (20_* §4). (d) Re-scope the 7 single-global-tenant assumptions (project/group name, device hardware_id, release monotonicity, active-deploy uniqueness, `CallerID` → `user_id`, token tenant dimension) (20_* §2.2).
- **Dependencies.** None upstream (this is the root). **Decision-gate inputs it needs first:** OQ-4 migration path (§5) and the TEXT-vs-UUID id-type divergence (20_* §1).
- **Tests that prove it (27_*).** Data-model/RLS surface (27_* §1): unit (memory) + integration (BOTH backends) + the **RLS-isolation** flagship layer A (27_* §2.1, pgx-only, live Postgres via rootless podman §11.4.161 — else honest SKIP: `topology_unsupported`) + concurrency/race on the re-scoped monotonicity/active-deploy keys and `CreateAccountWithOwner` atomicity + the **migration/backfill** correctness matrix (27_* §5). Each isolation assertion is three-part (A’s rows present, B’s absent, mutation flips both — 27_* §2 shared design) with a paired §1.1 mutation that drops the scope and makes the test FAIL.
- **Exit criteria.** Both backends pass the app-layer scope suite; RLS layer green on the pgx path (or explicitly SKIP-with-reason where Postgres absent); the 002 → backfill → 003 migration is idempotent +

crash-safe with `COUNT(account_id IS NULL)=0` and `COUNT(outside __default__)=0` captured (27_* §5); composite-FK integrity holds. No cross-account read returns data. Evidence under `docs/qa/<run-id>/isolation_rls/ + .../migration/`.

M2 — AuthZ / token account-claim / super-admin · derives from 21_* (owns OQ-2) (+ 20_* §6)

- **Scope.** The authorization model over M1's entities: the **RBAC-first role+scope hybrid** (tenant-isolation predicate → RBAC matrix → thin attribute deny-override, evaluated in that fixed order — 21_* §1.2), the sign-in + account-selection exchange flow (21_* §2), token scoping (add the server-minted immutable `account_id` claim, device/CLI also `project_id` — 21_* §3), the L1-middleware/L2-scoped-store/L3-RLS enforcement stack (21_* §4), and the super-admin as a **global `is_super_admin` flag** (not a per-account role) bypassing scope via a **policy-based RLS bypass, never a `BYPASSRLS` role** (21_* §1.4/§4, 20_* §6).
- **Deliverables.** (a) Claims extended with `account_id` (+ device/CLI `project_id`), signing still config-only (§11.4.10; 21_* §3.2); legacy tokens without the claim are **fail-closed on scoped routes, super-admin exempt** (21_* §3.3 Option A). (b) The server-authoritative permission matrix promoted from the shipped client-side `RESOURCE_PERMISSIONS` grid (21_* §1.3). (c) `requireAccountAccess` + `requireSuperAdmin` middlewares (the analogues of `requireProjectAccess`/`requireRole`).
 1. The **single canonical role vocabulary** reconciliation (§11.4.186): add `super_admin` to the server `token.go` set (the new SSOT) and to the dashboard `Role` union, and fix the `OTA-Manager sidebar.tsx "developer"` literal that is not in the union (21_* §5). (e) Least-privilege defaults + revocation semantics (21_* §6).
- **Dependencies.** M1 (entities + membership lookups + `is_super_admin` column). **Decision-gate input:** OQ-2 permission-model shape (§5) — the RBAC-first hybrid is recommended; the reserved roles/permissions tables stay an additive escalation.
- **Tests that prove it (27_*).** Server-API surface (27_* §1) + the **super-admin test set** (27_* §4): bypass works only for super-admin; the RLS bypass is policy-based + immediately revocable (clear the GUC → session re-scopes same request); audit written with the affected `account_id`; no request-side escalation (forged `is_super_admin` claim/field ignored). Each with a paired §1.1 mutation (grant bypass to a non-super-admin / make the bypass survive GUC-clear / drop

account_id from the audit write / honor a request-supplied is_super_admin) that makes the gate FAIL.

- **Exit criteria.** Scoped tokens authorize only their account; super-admin bypass is scoped-audited-revocable with captured GUC-on/ GUC-off row-set evidence (docs/qa/<run-id>/superadmin/); the role vocabulary is single-source across server + both SPAs; legacy tokens fail-closed. No forged-claim escalation succeeds.

M3 — Account-scope every OTA route + super-admin admin API · derives from 22_* (+ 20_*/21_*)

- **Scope.** Wire the model + authZ onto the HTTP surface: make the existing OTA routes account-scoped via the **token claim on the hot path** (paths unchanged), add the **super-admin admin API** (accounts/users/memberships/roles) as an additive /admin/* namespace gated by requireSuperAdmin, add the **account-selection API** (GET /auth/accounts + POST /auth/select-account token exchange), and the **credentials API** (API-key mint/list/revoke + POST /auth/token-exchange) (22_* §1–§4). Cross-account operations use **path tenancy for admin only** (22_* §2.0).
- **Deliverables.** (a) requireAccountAccess wired onto every currently-unscoped device/artifact/release/deployment/group/audit/client route (22_* §2.1), scope derived from the verified claim, project validated against the claim's authorized set — never widened. (b) The /admin/* route table (22_* §1.2). (c) GET /auth/accounts + POST /auth/select-account (22_* §3). (d) The **missing GET /artifacts (list) endpoint** the SPA + CLI block on (22_* §5.1). (e) /projects becomes account-scoped; cross-tenant denial returns **NOT_FOUND** (anti-enumeration) reusing the existing ErrorBody envelope + closed code set — no new codes, no /api/v2 (22_* §6). (f) Legacy-token default-account fallback for a bounded transition, then Sunset header (22_* §6.1) — composed with the OQ-4 Option-A backfill.
- **Dependencies.** M1 (scoped store) + M2 (the claim + middlewares + super-admin gate).
- **Tests that prove it (27_*).** The **flagship cross-account isolation** at the API layer (27_* §2.2, both backends — the memory MVP's only isolation is L2 here): scoped list returns only the caller's account; a known cross-account id is denied (404 vs 403 choice deferred to 26_*); a cross-project/-account key on write is 403; account is taken from the resolved scope, never a request field. Paired §1.1 mutation removes requireAccountAccess (or the AccountID filter) → A sees/ mutates B → FAIL. Plus DDoS (auth/upload flood per-account rate posture), concurrency/race, benchmarking (scope- resolution overhead) per 27_* §1.

- **Exit criteria.** Every OTA route rejects cross-account access at the API layer on both backends with captured HTTP transcripts + before/after store dumps (docs/qa/<run-id>/isolation_api/); the admin + selection + list-artifacts endpoints exist and are super-admin/scoped-gated; /api/v1 stays backward-compatible (additive).

M4 — UI: sign-in split + account switcher + super-admin console · derives from 23_* (+ §11.4.162 / §11.4.170)

- **Scope.** Extend the two in-scope SPAs — the operator **dashboard** (dashboard/) and the **OTA Manager** (clients/ota-manager/, doubling as the server /manager web console) — with: the **post-auth routing split** (super-admin → global console; user → account selection — 23_* §1.2 Option A recommended), the **post-login account picker** (auto-select single, persist-last, always-available switcher — 23_* §2), the **AccountSwitcher** mounted above the existing ProjectSwitcher (two-level context — 23_* §2.2), account-scoped cache keys + re-scope-on-switch (23_* §2.3), and the **super-admin console** (accounts/users/memberships/permissions matrix — 23_* §3).
- **Deliverables.** (a) OTA Manager: activeAccountId + accounts[] in auth-store, a new AccountSwitcher mirroring project-switcher.tsx, a use-accounts.ts hook, the /admin/* route + isSuperAdmin-gated nav, the src/features/admin/ screens, the permissions-matrix UI over the RESOURCE_PERMISSIONS shape (23_* §6.1). (b) Dashboard: add super_admin to its Role union FIRST (§5/23_* §5 — the console cannot gate without it), port the switcher/picker pattern, finish the 58-hardcoded-hex token vendoring before layering new screens (23_* §6.2). **MANDATORY:** every new/changed element consumes the helix-ota **OpenDesign tokens (§11.4.162)**, light + dark, no hardcoded hex, no label overlap.
- **Dependencies.** M2 (role vocabulary + super_admin + the account claim/exchange the UI state mirrors) + M3 (the real GET /accounts / /admin/* / account-selection endpoints the switcher/console call). **Decision-gate input:** OpenCode-vs-OpenDesign confirmation (§5) — does not block token guidance (targets the already-vendored tokens).
- **Tests that prove it (27_*).** **Device-independent host-rendered pixel proof (§11.4.170)** per screen × state × {light, dark}, **dual-validated** by golden image-diff AND the OCR/vision layout oracle (27_* §3) — value/token-equality tests are FORBIDDEN as the sole proof. The harnesses ALREADY EXIST (clients/ota-manager/visual/ with oracle-diff.mjs + oracle-ocr.mjs; dashboard/hostrender/ Playwright specs) — the tenancy screens add fixtures (27_* §3.1).

Oracles self-validated golden-good/ golden-bad (§11.4.107(10), 27_* §3.3). Plus the wire proof that switching account re-scopes the project list + invalidates caches (27_* §1 OTA-Manager row).

- **Exit criteria.** Every tenancy screen renders correctly per state × theme, dual-oracle GREEN with PNG + OCR JSON evidence (docs/qa/<run-id>/); switching account re-scopes data (captured); dashboard super_admin union landed; no hardcoded hex in new screens.

M5 — Project-side integration CLI (helix-ota) · derives from 24_* (+ 22_* §4/§5, 26_* §3)

- **Scope.** A production project-integration CLI — a single static Go binary (server/cmd/helix-ota/, beside ota-server) — that a consuming project's CI/CD uses to authenticate to **one account + one project** and publish OTA updates, **extending** the existing upload/publish mechanism (24_* §0), not reinventing it.
- **Deliverables.** (a) Non-interactive auth: an account+project-scoped, revocable, hash-stored **API key** used as an **exchange credential** (24_* §1.3 Option B recommended) — helix-ota login --token \$KEY → POST /auth/token-exchange → short-lived scoped bearer. (b) The command surface (login/whoami/config set-context/ upload/ artifacts list/release create/deploy create/status/doctor) with a kubectl-style local context, reusing the server's wire.go structs (24_* §2). (c) §11.4.10 no-leak: env HELIX_OTA_TOKEN primary, chmod 600 credential file, redaction to helixk_...<last4>, .gitignore fragment + .env.example (§11.4.77), a doctor token-leak audit. (d) The consuming-project incorporation recipe (24_* §5 — the snippets are the consumer's pipeline; no .yaml is added to helix_ota per §11.4.156).
- **Dependencies.** M3 (the account/project-scoped upload/release/ deploy routes + the API- key/token-exchange endpoints + the GET / artifacts list endpoint). **The e2e layer additionally depends on M7's object-storage Storage seam** (see §3): until it lands, helix-ota can authenticate/validate/register/release/deploy, but “device downloads + applies the exact bytes” is SKIP-with-reason: object_storage_absent, never faked (24_* §4/§6).
- **Tests that prove it (27_*).** CLI surface (27_* §1): unit (flag/arg parse, **token redaction** §11.4.10, exit-code map — mocks OK here only) + integration (real server: artifact persists under the correct (account_id, project_id); a **cross- project key is rejected 403**; a cross-account key cannot list another account's artifacts) + e2e (real server + real object storage + Go device emulator — blocked on M7) + stress+chaos (concurrent cross-account uploads with no bleed, kill-mid-upload) + full-automation -count=3 (§11.4.98) + a HelixQA

Challenge bank entry. Cross-project/ -account negative matrix is a first-class suite (24_* §6).

- **Exit criteria.** Integration + security layers GREEN with captured server JSON + store dumps + a redaction proof (raw token never in any log line); the e2e layer explicitly SKIP-with-reason until M7 — the coverage ledger is honest, not green-over-a-gap (24_* §6 recommendation).

M6 — Device/System-side update client + setup wizard · derives from 25_* (+ §11.4.162 / §11.4.170, 26_* §4)

- **Scope.** Extend submodules/ota-android-agent (today a headless 15-min poll worker, 11_* §4) to multi-account production: a server-minted (**account, project, device**) **identity** so a device only ever sees its own account's offer; a **hybrid transport** (poll stays the reliable floor + an optional push “wake” that carries no payload — never push-as-source-of-truth, 25_* §2.3); a per-account/project **silent/interactive notification policy**; and the **net-new setup wizard** (notify → wizard → consent → download → verify-before-apply → update_engine.applyPayload A/B handoff → reboot-to-switch with auto-rollback, 25_* §3/§4).
- **Deliverables.** (a) Device token gains server-minted account_id/project_id (client can never assert its own tenancy — the resolvePublicKey trust boundary, 25_* §1.2); ActiveDeploymentForTarget gains the account+project filter. (b) Provisioning: **Option A operator-minted scoped token** first, Option B bootstrap → rotate when self-service/fleet-scale is required (25_* §1.3). (c) The concrete ControlPlaneClient/Downloader/Verifier/Telemetry + device config that 11_* §Honest-gaps 2 says are absent — token in Android Keystore, config by injection (never hardcoded, §11.4.28), offline availability-following (§11.4.144), per-account signature verify key provisioned from server config (25_* §5). (d) The wizard as a net-new Compose UI on the :android layer — **OpenDesign tokens (§11.4.162)** light+dark + **Roborazzi/Paparazzi host-render proof (§11.4.170)**; kiosk silent path unchanged. A small server addition: a release notes field passed through the UpdateAvailable offer (25_* §3).
- **Dependencies.** M1–M3 (scoped device token + scoped update-check). **The byte-level download-and-apply e2e depends on M7's object-storage Storage seam + signed URLs (25_* §5 #6)** — until then the routing/offer isolation is provable, byte-apply is not.
- **Tests that prove it (27_*).** Device surface (27_* §1): unit (core Kotlin — VerifyBeforeApply order, wizard state machine, offline/resume) + integration (real ota-server) + the **device-layer flagship**

isolation (27_* §2.3: device A gets A's offer and 204/deny for B's deployment even at matching (os, model, group); paired §1.1 mutation reverts to the global key → device A offered B's deployment → FAIL) + UI host-render (§11.4.170) + stress+chaos (offline → resume, corrupt → reject, power-loss → A/B rollback) + the push-wake latency test.

- **Exit criteria.** Cross-account device isolation proven with captured poll transcripts (docs/qa/<run-id>/isolation_e2e/); the wizard renders correctly per state × theme (host-render dual-oracle); byte-apply e2e SKIP-with-reason until M7; the headless kiosk path is unregressed.

M7 — Security hardening + object storage · derives from 26_* (+ 24_* §4, 11_* §Honest-gaps)

- **Scope.** Close the 26_* threat controls that are not already covered by M1–M3's isolation, and land the **object-storage seam** that M5/M6 e2e depend on. **Sequencing note (§3):** the object-storage seam is a *prerequisite* the operator may pull EARLIER than the rest of M7 — the security-hardening items can trail, but the Storage seam gates every byte-level e2e claim (27_* §0.2).
- **Deliverables.** (a) **Object storage:** a Storage interface seam (like store.Repository) — dev uses rootless-podman **MinIO** (§11.4.161 via the containers submodule), prod S3/GCS; stream accepted bytes to durable storage, persist the **real** StorageRef (replacing the handlers_artifact.go:184 placeholder), **per-account bucket/key-prefix isolation**, and **signed, time-bounded download URLs** with the account_id/project_id derived from the verified device token (26_* §5, 24_* §4).
 1. **Per-account signing-key registry:**
resolvePublicKey(accountID) from a config-/ KMS-backed registry with a global fallback for un-migrated accounts — **the verify key still comes only from server config/registry, never the request; only the lookup key gains an account dimension** (26_* §4). Rotation with an overlap window; TUF-style M-of-N flagged as the hardening horizon (§11.4.112, not claimed shipped). (c) **The predictable dev-secret fix:** fail-fast (refuse to start) when HELIX_TOKEN_SECRET is unset in a multi-account build — the single highest-severity as-is finding (26_* §3.4/ §6). (d) Credential hardening: argon2id/HMAC-keyed hashing, shortened device-token TTL, §11.4.10.A pre-store leak audit before any secret lands.
- **Dependencies.** M1 (scoping + RLS) for the isolation controls; M5/M6 consume the Storage seam + signed URLs + per-account key.

- **Tests that prove it (27_* + 26_* proofs).** The DB-layer forged-WHERE RLS proof (26_* §1.3, 27_* §2.1); the super-admin no-BYPASSRLS/revocability proof (26_* §2.4, 27_* §4); credential redaction + expiry/revocation; the per-account signing-key negative test (A's key verifies for A, FAILs for B; a request-supplied verify key is structurally impossible — 26_* §4.4); the object-storage cross-account byte-access + signed-URL IDOR controls (26_* §5). Once the Storage seam lands, the M5/M6 byte-level e2e SKIPS flip to captured PASS (downloaded artifact hash matches the published one).
- **Exit criteria.** Fail-fast on unset secret; per-account key registry with rotation + global fallback; real per-account object storage with signed URLs; the byte-level e2e isolation proofs GREEN (no longer SKIP); every control carries its paired-mutation proof.

M8 — Full test sweep + manual-QA final gate · derives from 27_* (+ §11.4.40 / §11.4.185)

- **Scope.** The §11.4.40 **complete retest from a clean baseline** after every prior milestone is done + individually verified — NOT a spot-check of touched tests. This is the release-readiness gate for the whole work-stream.
- **Deliverables.** (a) The full 27_* test-type set across all six surfaces run GREEN with captured evidence, including the flagship three-layer isolation (RLS + app-scope + e2e, 27_* §2), super-admin (§4), migration/backfill (§5), and UI host-render (§3). (b) The 27_* §6 anti-bluff discipline enforced: ab_pass_with_evidence everywhere, every gate with a paired §1.1 mutation that FAILs, -count=3 determinism, no fakes beyond unit, the real PostgreSQL + real ota-server + real device emulator + real MinIO. (c) The completed **coverage ledger** (§6 below) with every row classified, the two by-construction SKIPS now resolved (Postgres present, object storage landed). (d) Independent verification/review of the test code itself iterating to a zero-finding GO (§11.4.142/§11.4.165/§11.4.134).
- **Dependencies.** M1–M7 all done + individually verified (§11.4.40 precondition).
- **Tests that prove it.** The entire 27_* suite; risk-ordered per §11.4.132 (most- recently-worked + most-problematic + highest-blast-radius + most-reopened first — the isolation + super-admin sets lead).
- **Exit criteria.** Every automated gate GREEN with captured physical evidence; the coverage ledger has no unresolved OPERATOR_ATTENDED_ONLY; then — the FINAL step (§11.4.185) — the **QA team's manual testing confirms** the feature. Only after that confirmation is the work-stream tag-eligible + merge-to-main-

eligible (§1.3). The agent hands off and waits for manual QA; it never self-certifies that step.

3. Critical path, danger zones, and risks (§11.4.172)

3.1 Critical path

M1 (data model + RLS) → M2 (claim + super-admin) → M3 (scoped routes + admin API) is a **hard serial spine** — each strictly depends on the prior. From M3, three tracks fan out and can run in parallel (§11.4.58/§11.4.103, disjoint file scope): **M4 (UI)**, **M5 (CLI)**, **M6 (device)**. **M7's object-storage seam is the load-bearing cross-cut**: it does not sit “after” M5/M6 — its **Storage seam is a prerequisite** for the byte-level e2e proofs of BOTH (24_* §4, 25_* §5, 27_* §0.2). **Recommendation**: pull the M7 object-storage seam EARLY (right after M3, in parallel with M4) so M5/M6 e2e are not left in permanent SKIP; the rest of M7's hardening can trail. M8 (full retest + manual QA) gates the merge. The longest path is therefore M1→M2→M3→M7(storage seam)→M5/M6 e2e→M8.

3.2 Danger zones (irreversible or high-blast-radius decisions)

- **Irreversible schema decisions (M1)**. The `account_id` NOT-NULL + **composite-FK + RLS 003 cutover** and the **TEXT-vs-UUID id-type** choice (20_* §1) are expensive to reverse once data exists. Mitigation: the 002(nullable) → backfill → 003(NOT NULL) sequence is designed to make the invariant clean on day one (20_* §5 Option A); every destructive migration step carries a §9.2 hardlinked backup + a defined expected post-op state + a restore-on-fail gate; target/ System safety per §11.4.133. This is the top danger zone precisely because M1 is the root of the serial spine.
- **The predictable dev-fallback HMAC secret (26_* §3.4 / §6 Spoofing)**. The token secret falls back to the literal "helix-ota-dev-token-secret-change-me" when `HELIX_TOKEN_SECRET` is unset (config.go:180-184). Under multi-account this is **catastrophic** — a predictable signing key lets an attacker mint a token with ANY `account_id` claim and defeat tenant isolation entirely. **This is the single highest-severity as-is finding**. Mitigation: M7's fail-fast (refuse to start unset) — but flagged here because it is a danger

zone the moment M2 introduces the account claim, earlier than M7; consider pulling the fail-fast into M2.

- **The RLS-needs-Postgres gap (27_* §0.1).** RLS (the database-enforced belt) exists ONLY on the pgx path; the default store is in-memory (`main.go:80`). On the memory MVP backend, L1+L2 (middleware + compile-time `explicit-accountID` scope) are the WHOLE isolation guarantee. Danger: a test run without Postgres proves only L1+L2; the RLS layer is an honest SKIP: `topology_unsupported`, never PASS-by-default. Mitigation: boot a real PostgreSQL on-demand via the containers submodule (rootless podman §11.4.161) for every RLS/migration test; the coverage ledger marks the SKIP explicitly.
- **The object-storage gap (11_* §Honest-gaps 1 / 26_* §5).** Artifact bytes are validated then **discarded** — `StorageRef` is a placeholder and no real object store is in-repo. Consequence: an end-to-end “device downloads + applies the exact bytes” cannot be proven from in-repo code until M7’s Storage seam lands. Danger: any “full e2e” claim before then would be a §11.4 PASS-bluff. Mitigation: M5/M6 e2e byte-apply is SKIP-with-reason: `object_storage_absent` until M7; the Storage seam is pulled early (§3.1); MinIO dev backend behind the Storage interface (§11.4.161).
- **Role-vocabulary divergences (21_* §5).** The same role vocabulary lives in FIVE places and they diverge: the `server token.go` set omits `super_admin`; the dashboard Role union omits `super_admin` (so the super-admin console cannot be gated there); the OTA-Manager `sidebar.tsx` gates on a “developer” literal that is not in the union at all (a latent gate bug — the class already bit). Danger: a divergent role literal is a latent gate bug; the dashboard super-admin UI is BLOCKED on the union fix. Mitigation: M2 makes the `server token.go` set the single SSOT and reconciles both SPAs in one sweep BEFORE any super-admin UI lands (M4 dashboard fixtures depend on it — 27_* §3.2).

3.3 Other tracked risks

- **The rollout subsystem’s tenancy is UNCONFIRMED:** (`internal/rollout/`, 10_* Honest-gaps / 26_* §Honest-boundary 4) — its store was not audited; on the evidence read it takes deployment ids, not account ids, but this is an unscoped path this plan cannot certify. **Action:** audit `rollout/store.go` before scoping its routes (an early M1/M3 sub-task, not deferred silently).
- **Memory-backend isolation rests entirely on L1+L2 (no RLS)** — the `explicit-accountID` compile-time param (chosen over `ctx-implicit`

precisely for this reason, 20_* §3.2 / 26_* §1.2) is what makes the default backend safe; 27_* §2.2 proves both backends.

- **Legacy-token / migration window** — the default-account fallback keeps a `__default__`-scoping path alive during transition; the Sunset-header deprecation (22_* §6.1) and the Option-B NULL-scoping guard (20_* §5, 27_* §5 Option-B variant) bound the isolation caveat.

4. Timeline projection framing (velocity-based; estimates are estimates — §11.4.6 / §11.4.172)

§11.4.172 requires realistic timeline projections from MEASURED velocity. This work-stream has **no measured velocity yet** (it is unstarted, and the plan is approval-gated), so **absolute calendar dates would be false precision** and are deliberately NOT asserted (§11.4.6). Instead this section gives (a) relative sizing per milestone, (b) the projection METHOD to apply once velocity is measured, and (c) the critical-path ordering — all marked as estimates.

Relative sizing (T-shirt, ESTIMATE — not calendar time):

Milestone	Relative size (estimate)	Why (dominant cost)
M1 data model + migrations + store	XL	keystone; touches every OTA table + both store backends + the 002/003 migration + RLS + the migration/backfill test matrix
M2 authZ + token claim + super-admin	L	claim + two middlewares + the policy-based RLS bypass + role-vocabulary reconciliation
M3 scope routes + admin API	L	every OTA route gated + the /admin/* + selection + list-artifacts endpoints +

Milestone	Relative size (estimate)	Why (dominant cost)
		both-backend isolation tests
M4 UI (2 SPAs)	L	switcher + picker + super-admin console + permissions matrix, ×2 surfaces, all host-render-proven light+dark
M5 project CLI	M	new binary but reuses wire.go; the hard part (object storage) is M7
M6 device client + wizard	XL	net-new concrete client + net-new Compose wizard UI + push-wake + per-account key verify + A/B handoff + host-render proof
M7 security hardening + object storage	L	Storage seam + MinIO + per-account key registry + fail-fast + signed URLs
M8 full retest + manual QA	M–L (elapsed, mostly machine + QA wait)	§11.4.40 is typically 12–48 h elapsed; plus the §11.4.185 manual-QA hand-off wait (not agent-controllable)

Projection method to apply once velocity exists (§11.4.172): (1) after M1–M2 land, measure actual effort per completed workable item to derive a velocity; (2) project the remaining milestones as **ranges** (optimistic / expected / pessimistic), never a single date; (3) re-project **monthly OR on any ≥10% item-count change** (§11.4.172); (4) treat the M1 serial spine (§3.1) and the M7-storage-seam prerequisite as the schedule’s fixed constraints; (5) the M8 manual-QA wait (§11.4.185) is an external dependency the agent cannot compress — model it as a

hand-off, not billable agent time. **All figures above are estimates and will be replaced by measured-velocity ranges; do not treat the T-shirt sizes as commitments.**

5. Operator-decision gate — resolve BEFORE implementation (§11.4.66 / §11.4.6)

This plan is APPROVAL-GATED. The following decisions MUST be resolved before the corresponding milestone starts — each is presented here with the design set’s **recommended option**, never silently decided. These consolidate into the forthcoming **28_* open-questions/decisions doc** (referenced per the task; not yet authored — verified absent this session), which will be the single decision-record surface.

#	Decision	Blocks	Recommended option (from the design docs)	Tradeoff to accept
OQ-2	Permission-model shape (RBAC vs ABAC vs role+scope hybrid)	M2	RBAC-first role+scope hybrid (tenant-isolation predicate → RBAC matrix → thin attribute deny-override), ABAC-capable by growth via the reserved roles/permissions tables — 21_* §1.2	fixed 3-role enum caps flexibility at first ship; custom-per-account roles are a later additive escalation (no 2nd data-model change)
OQ-3	Identity source (local super-admin-provisioned users vs	M1/M2	Local accounts only for now — the mandate implies local super-admin-provisioned	federation (OIDC/SAML) is a future addition, not built now

#	Decision	Blocks	Recommended option (from the design docs)	Tradeoff to accept
OQ-4	federated OIDC/SAML)		users; no self-registration (00_INDEX §1.2/ §5, 10_* §7)	
	Migration/backfill path for existing data	M1	Option A — default-account big-bang backfill, then NOT NULL (removes the nullable-isolation-hole window; the as-is store is in-memory-by-default + barely populated, making backfill nearly free) — 20_* §5 OpenCode = the coding AGENT used to build UI; OpenDesign = the mandated design-TOKEN system the UI consumes (§11.4.162) — both true, not in conflict; evidence-resolved in 12_* §3 (3299 OpenCode hits are all an LLM	requires a coordinated 003 cutover; flips to Option B ONLY if a target already holds meaningful prod data AND zero-downtime is mandatory
OpenCode vs OpenDesign	Confirm the mandate’s literal “build UI with OpenCode”	M4		flagged for confirmation only because the mandate used the literal word “OpenCode”; does not block M4 token guidance (targets the already-vendored helix-ota/tokens.css)

#	Decision	Blocks	Recommended option (from the design docs)	Tradeoff to accept
			coding-agent adapter; zero in any UI dir)	
Per-account signing keys	Global key vs per-account key registry	M7 (device/ CLI verify)	Per-account ed25519 key in a server-side registry, global key as fallback during migration — the verify key still comes ONLY from server config/ registry, never the request; only the lookup key gains an account dimension — 24_* §3 / 26_* §4	a registry lookup + per-account key lifecycle vs one global key; a shared global key lets any account's signer sign for the platform (unacceptable long-term for tenant isolation)

Secondary decisions also for 28_* (recommendation noted, not blocking a whole milestone): the **id type** TEXT-vs-UUID (20_* §1 — recommend TEXT to mirror the shipped store); the **sign-in split** Option A (post-auth routing) vs Option B (separate super-admin door) (23_* §1.2 — recommend A now, B open as hardening); the **CLI auth** direct-key (A) vs token-exchange (B) (24_* §1.3 — recommend B); the **device enrollment** operator-minted (A) vs bootstrap → rotate (B) (25_* §1.3 — recommend A now, B at scale); the **cross-tenant denial** NOT_FOUND vs FORBIDDEN (22_* §6.2 / 26_* — recommend NOT_FOUND anti-enumeration, hard rule owned by 26_*); the atomic-bootstrap seam CreateAccountWithOwner (A) vs generic WithTx (B) (20_* §3.3 — recommend A now).

6. Test-type coverage ledger skeleton (§11.4.25 / §11.4.169 / §11.4.52)

Feature × surface × test-type × status. **Status vocabulary** (§11.4.52 + 27_* §6): PLANNED (design-only, nothing built — the state of every row today), later AUTONOMOUS_DESIGNED → AUTONOMOUS_VERIFIED (captured evidence), OPERATOR_ATTENDED_ONLY (release blocker until promoted), NOT_APPLICABLE, or SKIP: <reason> for the two by-construction gaps (topology_unsupported = no live Postgres; object_storage_absent = no real object store). **Every row is PLANNED at this revision** — the feature is a to-be proposal; this skeleton is filled in as M1–M8 land, each cell citing its docs/qa/<run-id>/ evidence path. The six surfaces + closed test-type set are from 27_* §1.

Surfaces: SVR = Server API · DATA = Data model/RLS · CLI = helix-ota · DEV = device client · DASH = dashboard SPA · MGR = OTA-Manager SPA.

Test type (§11.4.169 closed set + §11.4.170)	SVR	DATA	CLI	DEV
unit	PLANNED	PLANNED	PLANNED	PLANNED
integration (real server + real store)	PLANNED	PLANNED (both backends)	PLANNED	PLANNED
e2e	PLANNED	—	SKIP:object_storage_absent → PLANNED@M7	SKIP:object_stor → PLANNED@M
full- automation (§11.4.98)	PLANNED	PLANNED	PLANNED	PLANNED
Challenges / HelixQA (§11.4.27)	PLANNED	PLANNED	PLANNED	PLANNED
DDoS	PLANNED	—	PLANNED	PLANNED

Test type (§11.4.169 closed set + §11.4.170)	SVR	DATA	CLI	DEV
security / tenant- isolation (FLAGSHIP)	PLANNED	PLANNED (RLS)	PLANNED	PLANNED
stress + chaos (§11.4.85)	PLANNED	PLANNED	PLANNED	PLANNED
concurrency / race-deadlock	PLANNED	PLANNED	PLANNED	PLANNED
memory	PLANNED	PLANNED	—	PLANNED
benchmarking	PLANNED	PLANNED (RLS vs no-RLS baseline)	PLANNED	—
UI host- render (§11.4.170)	—	—	—	PLANNED (Comp Paparazzi)
migration / backfill	—	PLANNED (pgx)	—	—

Two rows are honestly non-green by construction today (§0 of 27_*), **tracked in the ledger, never faked green:** the DATA/RLS layer is SKIP:topology_unsupported without a live PostgreSQL (M1/M7 boot it via rootless podman §11.4.161), and the **byte-level e2e** on CLI + DEV is SKIP:object_storage_absent until M7's Storage seam lands. Every SKIP carries its reason; a bare ab_pass is deprecated → FAIL (§11.4.69); each gate ships a paired §1.1 mutation that must make it FAIL (27_* §6); the whole ledger is reviewed by an independent verifier iterating to a zero-finding GO (§11.4.142/§11.4.165) before the §11.4.185 manual-QA final confirmation.

Honest boundary (§11.4.6)

- **Plan-only, approval-gated, not implemented.** This is a delivery PLAN synthesizing the design set (00_INDEX + 10_/11_/12_* as-is + 20_*-27_* to-be). No production code, migration, endpoint, UI component, CLI, device client, or test described here exists yet — the entire multi-account feature is a to-be proposal, and no work starts until the operator approves this plan AND resolves the §5 decision gate (§11.4.66). This plan is not a §11.4 completion claim.
- **Estimates are estimates (§11.4.6 / §11.4.172).** §4 gives relative T-shirt sizing and a projection METHOD, NOT calendar dates — there is no measured velocity yet, and asserting dates would be the false precision §11.4.6 forbids. The sizes are estimates to be replaced by measured-velocity ranges (optimistic/expected/pessimistic), re-projected monthly or on ≥10% item-count change (§11.4.172). No forbidden certainty vocabulary is used for any unproven claim.
- **SSOT deference (§11.4.186).** Entity/role/claim/API/UI/CLI/device/threat/test shapes are owned by their respective design docs (20_* is the data-model SSOT; 21_* owns OQ-2 + the token-claim shape; 22_* the routes; 23_* the UI; 24_*/25_* the CLI/ device; 26_* the threat model + the 404-vs-403 hard rule; 27_* the test strategy). This plan **sequences and cross-references** them; it invents no conflicting shape, and where an owner doc finalizes a different shape, the owner doc wins and this plan is reconciled to it.
- **The two by-construction gaps are real, not deferred silently.** RLS requires a live PostgreSQL (the default store is in-memory — memory-backend isolation rests on L1+L2 only); and “device downloads + applies the exact bytes” requires real object storage that is absent in-repo (bytes are validated then discarded, StorageRef is a placeholder). Both are surfaced in §3.2 (danger

zones), §6 (ledger SKIPS), and every milestone that touches them — the honest state is visible, never green-over-a-gap.

- **28_* (decision-consolidation doc) is not yet authored** (verified absent this session) — §5's decisions are recorded here with recommendations and will consolidate into 28_* as the operator resolves them. **The rollout subsystem's tenancy remains UNCONFIRMED:** (internal/rollout/, 10_* Honest-gaps) — a tracked pre-scoping audit, not certified by this plan.
- **No external research was performed for this plan** — it synthesizes the already- verified design set, whose own ## Sources verified 2026-07-10 footers carry the external citations. No new external claim is made here, so no ## Sources verified footer is added (nothing external is cited that the design set did not already verify).